

Vel Tech Group of Institutions

Name: KISHORE P P

Email: vtu27958@veltech.edu.in

Roll no: vtu27958

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS5L14

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
#include <stdio.h>
```

```
int n, target;  
int expenses[20];  
int subset[20];  
int found = 0;
```

```
void findSubsets(int index, int current_sum, int subset_size) {  
    if (index == n) {  
        if (current_sum == target && subset_size > 0) {  
            found = 1;  
            for (int i = 0; i < subset_size; i++) {  
                printf("%d%s", subset[i], (i == subset_size - 1) ? "" : " ");  
            }  
            printf("\n");  
        }  
        return;  
    }  
  
    subset[subset_size] = expenses[index];  
    findSubsets(index + 1, current_sum + expenses[index], subset_size + 1);  
  
    findSubsets(index + 1, current_sum, subset_size);  
}
```

```
int main() {  
    if (scanf("%d", &n) != 1) return 0;  
  
    for (int i = 0; i < n; i++) {  
        if (scanf("%d", &expenses[i]) != 1) return 0;  
    }  
  
    if (scanf("%d", &target) != 1) return 0;  
  
    findSubsets(0, 0, 0);  
  
    if (!found) {  
        printf("No subset matches the target sum\n");  
    }  
  
    return 0;
```

}

Status : Correct

Marks : 10/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: $v_1 v_2 v_3 \dots$ "

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

1 3

2 3

3 4

3

Output: Clique Found: 0 1 2

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int graph[20][20];
```

```
int nodes[20];
```

```
int n, m, k;
```

```
int found = 0;
```

```
int is_clique(int sz) {
```

```
    for (int i = 0; i < sz; i++) {
```

```
        for (int j = i + 1; j < sz; j++) {
```

```
            if (!graph[nodes[i]][nodes[j]]) {
```

```
                return 0;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

```
void find_clique(int start, int sz) {
```

```
    if (sz == k) {
```

```
        if (is_clique(k)) {
```

```
            found = 1;
```

```
            printf("Clique Found:");
```

```
            for (int i = 0; i < k; i++) {
```

```
                printf(" %d", nodes[i]);
```

```
            }
```

```
            printf("\n");
```

```
        }
```

```
    }
```

```
}
```

```

    for (int i = start; i < n; i++) {
        if (found) return;
        nodes[sz] = i;
        find_clique(i + 1, sz + 1);
    }
}

int main() {
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
    }

    for (int i = 0; i < m; i++) {
        int u, v;
        if (scanf("%d %d", &u, &v) == 2) {
            graph[u][v] = 1;
            graph[v][u] = 1;
        }
    }

    if (scanf("%d", &k) != 1) return 0;
    find_clique(0, 0);

    if (!found) {
        printf("No clique of given size exists\n");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate

containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: $x_1 x_2 x_3 \dots$
- Subset 2: $y_1 y_2 y_3 \dots$

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 11 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
#include <stdio.h>
```

```
int n;
```

```
int arr[16];
int included[16];
long long total_sum = 0;
int found = 0;
```

```
void print_subsets() {
    printf("Subset 1:");
    for (int i = 0; i < n; i++) {
        if (included[i]) {
            printf(" %d", arr[i]);
        }
    }
    printf("\nSubset 2:");
    for (int i = 0; i < n; i++) {
        if (!included[i]) {
            printf(" %d", arr[i]);
        }
    }
    printf("\n");
}
```

```
void solve(int index, long long current_sum) {
    if (found) return;

    if (current_sum == total_sum / 2) {
        found = 1;
        print_subsets();
        return;
    }

    if (index == n || current_sum > total_sum / 2) {
        return;
    }

    included[index] = 1;
    solve(index + 1, current_sum + arr[index]);

    if (found) return;

    included[index] = 0;
    solve(index + 1, current_sum);
}
```



```
int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        total_sum += arr[i];
    }

    if (total_sum % 2 != 0) {
        printf("No valid equal partition exists.\n");
    } else {
        solve(0, 0);
        if (!found) {
            printf("No valid equal partition exists.\n");
        }
    }

    return 0;
}
```

Status : Correct

Marks : 10/10